

PROJETO FINAL

PROGRAMAÇÃO E SISTEMAS DE INFORMAÇÃO – 2º ANO CURSO PROFISSIONAL TÉCNICO DE GPSI



“A perseverança é o caminho para o sucesso.”
(Charles Chaplin)



Módulo 11

Programação Orientada a Objetos Avançada



PROJETO: estás preparado?

Neste módulo, o desafio é consolidar os conhecimentos de Programação Orientada a Objetos (POO), aplicando conceitos avançados como herança, polimorfismo, composição e persistência de dados.

O objetivo é desenvolver uma aplicação Windows (WinForms) completa e robusta que resolva um problema real de gestão de dados, garantindo a qualidade do código, a usabilidade da interface e o trabalho colaborativo através de ferramentas de controlo de versões (Git/GitHub).



Definição do Tema

O tema do projeto é livre e escolhido pelos alunos. O tema deve retratar um cenário real de gestão de informação.

Exemplos de temas possíveis:

- Gestão de uma Clínica Veterinária (Animais, Consultas, Donos).
- Gestão de Aluguer de Equipamentos (Câmaras, Portáteis, Requisitantes).
- Gestão de uma Biblioteca ou Videoteca (Livros, Sócios, Empréstimos).
- Gestão de Oficinas (Veículos, Reparações, Mecânicos).
- Gestão de Eventos e Bilheteira.
- (...)

Nota: O tema deve ser validado pelo professor antes do início do desenvolvimento.



Requisitos Técnicos Obrigatórios

O projeto deve cumprir rigorosamente os seguintes requisitos para ser considerado para avaliação:

Estrutura de Classes e POO (Mínimo: 4 Classes)

O núcleo do programa deve ser uma **Lista de Objetos** que suporte todas as operações de gestão. A modelação das classes deve obrigatoriamente aplicar:

1. **Herança:** Existência de uma classe base e classes derivadas (ex: Veiculo -> Carro, Mota).
2. **Composição:** Pelo menos uma classe deve conter instâncias de outra classe ou listas de outra classe (ex: um Cliente tem uma Morada; uma Venda tem uma lista de Artigos).
3. **Polimorfismo:** Implementação de métodos que apresentem comportamentos distintos nas classes derivadas (uso de override, métodos virtual ou abstract).

Funcionalidades CRUD e Identificação

A aplicação deve garantir as operações CRUD (Create, Read, Update, Delete) sobre a lista principal:

- **Listar:** Visualizar os dados numa DataGridView ou ListBox.
- **Adicionar:** Inserir novos registos através de um formulário próprio.
- **Editar:** Alterar dados de um registo existente.
- **Remover:** Apagar registos (com mensagem de confirmação).

Requisito de Identificação (ID)

Todos os objetos da lista devem possuir uma propriedade ID (número inteiro) que os identifica inequivocamente. Esta numeração deve ser **automática** e gerida pelo sistema (o utilizador não digita o ID), garantindo que nunca existem dois objetos com o mesmo ID, mesmo após a eliminação de registos.

Validação de Dados

É proibida a entrada de dados incorretos ou inconsistentes.

- Deves utilizar o componente **ErrorProvider** nos formulários para validar campos obrigatórios, tipos de dados (números vs. texto) e limites aceitáveis.
- O botão de “Gravar” só deve ficar disponível ou funcionar se não existirem erros de validação.

Persistência de Dados (Ficheiros)

Os dados não se podem perder quando a aplicação fecha.

- Implementar métodos de **Leitura** e **Escrita** em ficheiro para guardar o estado da lista.
- Sugere-se o uso do formato **JSON**, permitindo guardar a estrutura hierárquica das classes.
- A aplicação deve carregar os dados automaticamente ao iniciar.



Funcionalidades de Majoração (Valorização)

Para atingir as cotações máximas (níveis de excelência), o projeto deve incluir funcionalidades avançadas, tais como:

- **Pesquisa e Filtros:** Permitir filtrar a lista por critérios (ex: mostrar apenas “Pendentes”, pesquisar por nome).
- **Ordenação:** Permitir ordenar a lista por diferentes colunas.
- **Painel de Estatísticas:** Apresentar totais, médias ou contagens (ex: “Total de vendas hoje”, “N.º de clientes ativos”).

- **Interface Gráfica:** Menus organizados, ícones, atalhos de teclado e feedback visual claro ao utilizador.



Metodologia de Trabalho: Git e GitHub

A competência “Comunicar e Colaborar” será avaliada através do uso do GitHub.

1. **Repositório:** Criar um repositório para o projeto.
Para garantir a integridade da avaliação e a privacidade do teu código, o repositório deve ser configurado corretamente desde o primeiro momento. Ao criá-lo, deves seleccionar obrigatoriamente a opção de visibilidade Private, uma vez que o projeto não pode estar público durante a fase de desenvolvimento. Adicionalmente, para que a equipa docente possa corrigir, acompanhar o histórico de commits e validar o trabalho realizado, é imperativo que dês acesso aos professores, adicionando-os como colaboradores nas definições do repositório.
2. **Commits Regulares:** O desenvolvimento deve ser progressivo. Não serão aceites projetos com apenas um “commit final”. Os commits devem ter mensagens descritivas do que foi feito (ex: “*Adicionada validação de NIF*”, “*Correção do bug no cálculo do preço*”).
3. **Ficheiro README.md:** O repositório deve ter uma capa (README) contendo:
 - Título e descrição do tema.
 - Autores (Turma, Nome e N°).
 - Lista de funcionalidades implementadas.
 - Instruções simples de como utilizar.



Etapas de Desenvolvimento e Entrega

O projeto será desenvolvido de forma incremental, seguindo as fases abaixo para garantir um acompanhamento contínuo e feedback frequente.

ETAPA 1: Planeamento e Configuração (Início)

- **Definição do Tema:** Escolha do cenário real e submissão da proposta ao professor para validação.
- **Configuração Git:** Criação do repositório no GitHub e escrita do README.md inicial com a descrição do projeto.

ETAPA 2: Desenvolvimento do Projeto (Execução)

- **Implementação do Modelo:** Criação das classes com as propriedades e métodos necessários.
- **Interface e CRUD:** Construção dos formulários WinForms para listagem, inserção e edição, garantindo a atribuição do ID Automático.
- **Validação e Persistência:**
 - Implementação de exceções (blocos Try/Catch);
 - Validação de Dados com ErrorProvider (o botão inscrever/inserir só deve funcionar se não existirem erros);
- **Implementação de métodos de leitura/escrita em formato JSON (básico);**
 - Gravar e ler lista de participantes (Eventos);
- **Implementação de métodos de leitura/escrita em formato JSON:**
 - Apenas os participantes que tenham idade ≥ 18 anos;
 - Guardar participantes inscritos na data atual (dia);
- **Controlo de Versões:** Realização de *commits* regulares no GitHub à medida que cada funcionalidade (ex: “Gravar em ficheiro”) é concluída.

ETAPA 3: Entrega Final

- O repositório no GitHub deve estar atualizado com a versão final estável (sem erros de compilação).
- Entrega da pasta do projeto compactada via Microsoft Teams contendo o link direto para o repositório GitHub.
- A pasta do projeto deve ser compactada e nomeada de acordo com o formato padrão: 11?-N??-NOME.zip

ETAPA 4: Apresentação e Defesa

- Apresentação da aplicação em funcionamento perante a turma/professor, percorrendo o fluxo completo (CRUD e Ficheiros).
- Resposta a questões sobre a estrutura do código, explicando o seu funcionamento.

ETAPA 5: Autoavaliação e Reflexão

- Avaliação do Trabalho: Realiza uma avaliação crítica do teu projeto, promovendo a reflexão sobre o percurso e o grau de responsabilidade nas aprendizagens realizadas.
- Análise Crítica: Identificação dos pontos fortes do projeto, dificuldades encontradas na utilização do Git/GitHub e propostas de melhoria futura.
- Questionário de autoavaliação: <https://forms.office.com/e/6TuiwWDAnj>

BOM TRABALHO!
ESTÁS NO CAMINHO CERTO PARA CONSTRUIR O TEU CONHECIMENTO!

Os professores da disciplina,
Andreia Quintal | Carlos Almeida | Cristina Gonçalves | Pedro Cardoso